
Decision Transformer Warm-Starts for Minimum-Time Vehicle Trajectory Optimization with Obstacle Avoidance

Victor Sebastian Martinez Perez ¹

Abstract

This project studies whether a Decision Transformer (DT) can provide useful warm-start trajectories for nonlinear minimum-time vehicle trajectory optimization with obstacle avoidance. The intended use case is not open-loop imitation alone, but reducing downstream nonlinear solver work by initializing the optimizer with a rollout conditioned on track geometry, obstacle context, and return-to-go. The final implemented system includes: (i) a direct-collocation trajectory optimizer, (ii) an autoregressive DT warm-start generator with dynamics-in-the-loop rollout, (iii) multiple repair-data pipelines including post-projection and failure-conditioned segments, and (iv) benchmark scripts with acceptance gating, projection diagnostics, and rollout traces.

Across successive training runs, offline validation improved substantially. The best parent model reached validation loss 0.002835, a lateral-weighted fine-tune reached 0.002773, and a failure-conditioned fine-tune reached 0.002364. However, these offline gains did not translate into a clear downstream win. Under the corrected evaluation path, warm-start acceptance remained near 0% under the default gate, iteration reduction remained 0%, and no DT variant clearly beat the baseline once warm-start inference time was included. A corrected relaxed-gate diagnostic did admit some runs (up to 20% acceptance on a 10-scenario gate), but those accepted cases were still not solver-helpful on average.

The final diagnosis is that the main remaining deployment bottleneck is lateral rollout behavior rather than optimizer integration alone. Early versions were dominated by fallback-heavy insta-

bility in u_y and r ; after several correctness fixes, the dominant failure shifted to early wrong-side or overly aggressive lateral branch selection that triggered repeated lateral clipping and rejection.

1. Introduction

This project investigates a pragmatic question: can a sequence model trained offline on nonlinear trajectory-optimization solutions provide warm-starts that reduce the cost of solving a minimum-time raceline problem with obstacle avoidance? The project focus is narrower than a generic reinforcement learning study. The relevant downstream objective is not imitation loss alone, but whether a DT-generated initial trajectory helps the nonlinear optimizer solve faster, with fewer iterations, or with higher robustness than the baseline initializer. The setup combines nonlinear vehicle trajectory optimization with Decision Transformer-style return-conditioned sequence modeling (??).

The final repo state reflects a progressively narrowed scope. The active execution loop is Oval-track only, uses FATROP-clean generated data for training shards, and uses fixed start-state assumptions for evaluation: the start progress s_0 varies along the centerline, while $e_0 = d\psi_0 = u_{y,0} = r_0 = 0$ and $u_{x,0} = 5.0$ m/s. Two benchmark families are used: a no-obstacle family and a frozen 1–4 obstacle family. In practice, IPOPT was used for most completed diagnostic evaluations because full fixed-gate FATROP evaluation was too slow for interactive iteration, while FATROP remained the generation path for the cleaned training data.

The main result is negative but informative. The implemented DT system learns the offline data well and successive data interventions do improve validation loss. However, those gains do not produce a convincing end-to-end warm-start benefit. This report therefore emphasizes what was actually implemented, what was measured, which confounds were removed, and what the final failure mode appears to be.

¹Department of Aeronautics and Astronautics, Stanford University, Stanford, CA, USA. Correspondence to: Victor Sebastian Martinez Perez <sebasmp@stanford.edu>.

2. Background and Related Work

The project combines two established ideas. First, minimum-time racing-style trajectory optimization is naturally posed as a nonlinear program over vehicle dynamics, track constraints, and obstacle-clearance constraints (??). Second, Decision Transformer-style sequence modeling treats control as conditional sequence prediction over returns, states, and actions (?).

The motivation for combining them is straightforward: if expert trajectories from a strong optimizer can be collected offline, then a transformer may learn a policy that produces a high-quality initial guess for the same optimizer at test time. In principle, even a partially correct warm-start could reduce solver iterations or solve time. This hybrid use of transformers as optimizer priors is close in spirit to prior transformer-based warm-starting work in trajectory optimization (?).

In this project, however, the relevant question is not whether DT can imitate expert trajectories in distribution, but whether the closed-loop rollout induced by autoregressive action prediction stays inside the dynamically valid and constraint-compatible regime needed for nonlinear warm-starting. That distinction became central in the final diagnosis.

3. Problem Setup

We study minimum-time trajectory optimization for a vehicle on a closed track with optional static circular obstacles. The optimizer uses a spatial discretization with horizon length $N = 120$ on the Oval track. The underlying state used by the optimizer is

$$x = [u_x, u_y, r, \Delta F_{z,\text{long}}, \Delta F_{z,\text{lat}}, t, e, d\psi],$$

with controls

$$u = [\delta, F_x].$$

The optimization objective is the track lap-time objective plus a small control regularizer. Track bounds and obstacle-clearance constraints are enforced along the spatial trajectory. The DT does not replace this optimizer. Instead, it predicts an initial state-control rollout that is either accepted and passed to the optimizer or rejected and replaced by the baseline initializer.

The core evaluation question is therefore:

Does the DT warm-start reduce downstream solver work relative to the baseline initializer, while preserving solve success?

4. Method

4.1. Planned Method

The original plan was to train a DT on optimizer-generated trajectories and then use autoregressive rollout to initialize the nonlinear solver. The hoped-for outcome was a warm-start that was already close to a feasible raceline around obstacles, reducing solver iterations and total solve time. Post-projection repair data and later failure-conditioned data were proposed specifically to address rollout distribution shift: once the DT leaves the expert manifold, optimizer-labeled corrections may provide the right training signal.

4.2. Actual Implemented Method

The implemented DT uses a causal transformer over triplets of return-to-go, augmented state, and action. The augmented state contains the vehicle/path observation

$$[u_x, u_y, r, e, d\psi, \text{pos}_E, \text{pos}_N, \psi_{\text{world}}],$$

track features ($\kappa, \text{half} - \text{width}$), and a fixed number of obstacle slots containing lookahead obstacle features. The main model predicts the current action, and training also uses an auxiliary next-state prediction head.

Training minimizes

$$\mathcal{L} = \mathcal{L}_a + \lambda_x \mathcal{L}_x,$$

where both losses are masked MSE terms normalized only over valid, non-padding tokens. This detail matters because the repo explicitly fixed masked-loss normalization and padded-window handling during the project. Later fine-tunes also used state-dimension weighting that emphasized u_y, r, e , and $d\psi$.

At inference time, the warm-starter rolls out the DT autoregressively. Each step:

1. builds the augmented observation from the current state, track geometry, and nearby obstacles;
2. predicts the next control action;
3. propagates dynamics to get the next state;
4. optionally projects the next state back toward a safe envelope; and
5. validates the resulting rollout before handing it to the optimizer.

The wrapper supports three projection modes: `off`, `soft`, and `full`. It also tracks fallback counts, projection fraction, total projection magnitude, max projection magnitude, and per-reason counts such as `e_clip` and `obs_push`. These diagnostics became central to interpreting downstream failures.

Table 1. Main DT runs in the final repo state. Validation numbers are best logged checkpoints from `metrics.jsonl`.

Run	Data Mix	Parent	Best Val Loss	Best Val Action Loss
Parent improved	shift 99.5%, hard 0.5%	0.10	0.002835	0.002600
Post-proj FT2 rerun	shift 80%, hard 5%, postproj 15%	0.10	0.003170	0.002868
Lateral FT	shift 75%, hard 5%, postproj 20%	0.05	0.002773	0.002544
Failure-conditioned FT	shift 70%, failure 20%, hard 5%, postproj 5%	0.05	0.002364	0.002197

4.3. Wrapper Behavior and Acceptance Gate

The repo’s deployment logic does not automatically trust any DT rollout. A generated warm-start is accepted only if it passes both geometric validation and a threshold-based gate. In the default diagnostic configuration, the gate rejects a rollout if any of the following hold:

- fallback count > 0,
- projection fraction > 0.8,
- projection total magnitude > 100,
- projection max-step magnitude > 2.0.

This gate is important for two reasons. First, it protects the optimizer from poor initializations. Second, it means that offline improvement alone is insufficient: the DT must produce rollouts that are good enough under the actual acceptance policy.

5. Data and Training Pipeline

5.1. Base Data

The final active training regime uses Oval-only FATROP-clean data. The documented dataset status is:

- shift shard: 31,710 episodes,
- hard-repair shard: 400 accepted episodes,
- post-projection repair shard: 1,000 accepted episodes,
- failure-conditioned repair shard: 300 accepted episodes.

The shift shard provides broad nominal coverage; the repair shards target exactly the off-manifold states that appear during DT rollout and wrapper correction.

5.2. Training Runs

Table 1 summarizes the main completed runs from the final repo state.

The parent improved run used a larger architecture than earlier baselines: context length 50, 6 layers, 4 heads, and embedding size 192. Fine-tune runs reused that architecture and resumed from the parent checkpoint.

Two observations matter. First, offline validation clearly improved over the course of the project, especially after failure-conditioned data was introduced. Second, these offline improvements did not reliably predict downstream warm-start quality.

5.3. Failure-Conditioned Repairs

The failure-conditioned pipeline is one of the most substantive repo contributions. The process is:

1. export rollout traces from current checkpoints,
2. identify failure buckets such as repeated `e_clip` and early lateral instability,
3. rebuild repair segments around those failures using the nonlinear solver, and
4. fine-tune on the resulting accepted segments.

The documented trace export produced at least two populated buckets:

- `repeated_e_clip`: 2,987 examples,
- `early_lateral_failure`: 162 examples.

The first completed failure-conditioned shard contained 300 accepted repair segments.

6. Evaluation Protocol

6.1. Primary Deployment Metrics

The report uses the repo’s downstream warm-start metrics rather than offline loss alone:

- solver success rate,
- solver iterations,
- solver solve time,
- total DT time = warm-start inference + solve time,
- warm-start acceptance rate,
- fallback count,
- projection fraction,
- projection total magnitude,

- projection max-step magnitude.

A central lesson of the project is that validation loss and deployment metrics must be separated. Several later checkpoints improved offline and still failed to improve solver-facing behavior.

6.2. Benchmark Scope

The final narrowed benchmark uses two frozen Oval gates:

- no-obstacle gate,
- 1–4 obstacle gate.

Most completed controlled evaluations use IPOPT rather than FATROP. This was a practical choice: FATROP remained the main generation path for clean training data, but IPOPT was used for fixed-gate diagnostic evaluations because it was faster and more reliable for repeated interactive benchmarking.

6.3. Important Evaluation Caveats

Three caveats are required for faithful interpretation.

First, an early no-obstacle evaluation was confounded by repeated starts. The evaluation path was effectively repeating the same start configuration, so an initial 10-scenario no-obstacle gate was not actually testing 10 distinct start anchors. This was later corrected.

Second, the `lap_time_s` field stored in evaluation outputs corresponds to the optimizer objective value rather than a pure terminal-time readout, so we do not use it as a primary KPI in this report.

Third, acceptance rates depend strongly on the rejection gate. A corrected relaxed-gate diagnostic is useful for understanding rollout quality, but it does not by itself show solver benefit.

7. Results

7.1. Offline Training Improved, But Deployment Did Not

The strongest offline checkpoint in the final repo state is the failure-conditioned fine-tune, which reduced validation loss from the parent run's 0.002835 to 0.002364. The lateral-weighted fine-tune also improved over the parent to 0.002773. By contrast, the post-projection FT2 rerun was worse offline than the parent (0.003170).

This ranking does not match deployment behavior. The repo's own analysis identifies `oval-fatrop-improved-postproj-ft2-rerun +`

`projection=soft` as the strongest DT operating point before the failure-conditioned run, even though it was not the best offline model. This is a useful negative result: the project found a meaningful gap between offline imitation metrics and warm-start utility.

7.2. Completed Projection Ablation

A completed 10-scenario-per-gate projection ablation compared the parent improved checkpoint and the post-projection FT2 rerun under `off`, `soft`, and `full` projection modes. The repo's final verdict was:

- best parent regime: `full`,
- best FT2 regime: `soft`,
- best DT operating point overall: `FT2 + soft`,
- no DT variant beat the baseline once warm-start inference time was included,
- acceptance remained 0% for all default-gate runs,
- iteration reduction remained 0% for all runs.

For example, on the 10-scenario obstacle gate, `FT2 + soft` had baseline solve time 54.48s, DT solve time 52.58s, but DT total time 55.21s with 0% warm-start acceptance. Similar behavior held on the no-obstacle gate.

7.3. Corrected Relaxed-Gate Diagnostic

After fixing the repeated-start confound, the corrected relaxed-gate diagnostic for `FT2 + soft` showed that acceptance was not literally impossible. On the corrected 10-scenario evaluation:

- no-obstacle gate: acceptance 20%, baseline solve time 49.06s, DT total time 51.98s, iterations 206.8 vs baseline 191.8;
- obstacle gate: acceptance 20%, baseline solve time 56.36s, DT total time 60.79s, iterations 256.0 vs baseline 230.7.

These runs are important because they separate two claims:

1. the default gate was partly suppressing acceptance;
2. even when acceptance is relaxed, the accepted DT rollouts were still not solver-helpful on average.

Table 2. Final post-fix smoke benchmark for oval_fatrop_failure_ft + projection=soft. Each gate has only 3 scenarios.

Gate	Baseline Solve	DT Solve	DT Total
No obstacle	47.91s	46.34s	49.04s
1–4 obstacles	64.85s	62.47s	65.18s

7.4. Final Post-Fix Smoke Benchmark

The final post-fix smoke rerun focused on the failure-conditioned model, after the key correctness bugs had been fixed. These are small benchmarks with only 3 scenarios per gate and should be interpreted as smoke tests, not definitive leaderboard results. Table 2 summarizes them.

These results are consistent with the overall project conclusion. The DT sometimes lowers raw solve time slightly, but once warm-start inference is included, total time is still not better than baseline. More importantly, warm-start acceptance remains 0%, and iteration reduction remains 0%.

7.5. Projection Burden in the Final Smoke Benchmark

The final failure-conditioned smoke rerun also shows why acceptance remained at 0%. On the no-obstacle gate, the mean projection total magnitude was 127.29, the mean max-step projection was 4.91, and the mean projection fraction was 0.628. On the obstacle gate, the corresponding values were 121.03, 4.90, and 0.544. These exceed the default gate thresholds even though fallback was already greatly reduced.

This is an important qualitative shift. Earlier failure patterns were dominated by fallback-heavy instability; the final post-fix runs often fail without much fallback at all.

8. Debugging and Diagnosis

8.1. What Was Fixed

The repo records several concrete correctness fixes that materially changed interpretation of earlier results.

Repeated-start evaluation confound. An early no-obstacle benchmark repeated the same start rather than testing distinct frozen anchors. After correcting start-progress variation, the no-obstacle relaxed-gate acceptance changed from 0% to 20% for FT2 + soft. This did not make the DT competitive, but it did invalidate the earlier interpretation.

Warm-start validation bug. Warm-start validation was checking against progress $s = k \text{ ds}$ while rollout generation used $s = s_0 + k \text{ ds}$. This could falsely label nonzero-start rollouts as violating track or obstacle geometry.

Missing padded-context attention mask. Training used an attention mask that hides left padding in short contexts, but early inference rollout was missing this mask. Since most failures occurred in the first K steps and $K = 50$, this was a serious train/inference mismatch.

Wrong-side obstacle push projection. When both left and right obstacle-clearance pushes were feasible, the projection could choose the wrong side. This could move the state through the obstacle rather than away from it.

8.2. Diagnosis Progression

The project diagnosis evolved in three stages.

Stage 1: early fallback / lateral instability. Trace instrumentation showed that many rejected rollouts triggered fallback at $k = 0$ or $k = 1$. The raw dynamics step often produced extreme hidden lateral states even when the visible trajectory looked benign after fallback. Representative values in the repo include u_y and r exploding to hundreds in magnitude. This pointed to weak learned lateral-dynamics stability rather than simple geometric drift.

Stage 2: projection-dominated rejection. After some wrapper stabilization, fallback pressure decreased, but many rollouts still accumulated large projection burden. The dominant projection reason was usually `e_clip`, sometimes followed by `dpsi_clip`. This suggested that the DT was still not maintaining a laterally consistent path under rollout.

Stage 3: wrong-side / wrong-branch lateral behavior. After the major correctness fixes, the dominant failure changed again. In the final failure-conditioned trace rerun:

- no-obstacle scenarios first hit `e_clip` at $k = 10$ in all 3 traced cases, with fallback count 0;
- obstacle scenarios first hit `e_clip` at $k = 13$ –15, with fallback 0–2.

Matched baseline-versus-DT comparisons showed that the DT often chose the wrong lateral branch early. In representative no-obstacle and obstacle cases, baseline steering kept the vehicle on the negative- e side while the DT steered toward positive e and crossed the soft boundary. In other cases the DT stayed on the same side but moved outward too aggressively.

This is the final diagnosis supported by the repo state: the dominant remaining deployment error is not simply that the model needs looser thresholds. It is that early lateral control selection is often wrong in sign or too aggressive in magnitude, and projection only exposes that mistake.

9. Discussion

The project produced two useful conclusions.

First, the engineering problem is real. A sequence model can achieve very low offline loss and still fail as a warm-start policy once it is rolled out closed-loop through nonlinear vehicle dynamics. The gap here was not hypothetical; it persisted across multiple dataset interventions and fine-tuning strategies.

Second, the repo identifies a fairly specific next bottleneck. The project no longer appears blocked primarily by generic model capacity or by a single missing wrapper heuristic. The final evidence points to early lateral branch selection under rollout as the core weakness. That finding is stronger than saying “DT is unstable.” It says the model often commits to the wrong lateral maneuver early, after which clipping and validation reject the trajectory.

The post-projection and failure-conditioned data interventions were still useful. They improved offline performance and helped expose which failure modes remained after major bugs were removed. The fact that accepted relaxed-gate rollouts were still solver-harmful is also informative: the objective should not be merely to increase acceptance under a wrapper gate, but to produce initializations that genuinely reduce solver work.

10. Limitations

This project has several limitations that materially affect interpretation.

- Many final downstream comparisons are small smoke benchmarks, especially the final post-fix failure-conditioned rerun with only 3 scenarios per gate.
- Most completed controlled evaluations are IPOPT diagnostics, not full FATROP deployment benchmarks, even though FATROP-clean data generation was the active dataset pipeline.
- Acceptance depends on a hand-designed threshold gate, and some projection metrics are aggregate wrapper diagnostics rather than physically clean control-theoretic quantities.
- The evaluation output’s `lap_time` field corresponds to optimizer objective value rather than pure terminal time, so it should not be over-interpreted.
- The final failure-conditioned shard contains only 300 accepted segments, which is enough for a targeted fine-tune but still small relative to the shift shard.

Because of these limitations, the strongest safe conclusion is not that DT warm-starting fails in general, but that the imple-

mented system in this repo did not yet produce a convincing solver-facing win on the final corrected benchmarks.

11. Conclusion

The final repo state supports a restrained conclusion. The project successfully implemented a DT-based warm-start pipeline for nonlinear minimum-time vehicle trajectory optimization, including rollout generation, repair-data production, and detailed downstream diagnostics. Several substantial bugs and evaluation confounds were found and fixed, and offline training improved meaningfully across successive runs.

However, the final measured deployment outcome is negative. Warm-start acceptance stayed near 0% under the default gate, iteration reduction stayed 0%, and no DT variant clearly beat the baseline once warm-start inference was included. A corrected relaxed-gate diagnostic showed that some rollouts could be admitted, but those accepted cases still did not reduce solver work on average.

The most useful final diagnosis is that the remaining bottleneck is early lateral rollout behavior, especially wrong-side or overly aggressive lateral branch selection. The next logical step is therefore not another blind scale-up, but a targeted data intervention around the first clip window ($k \approx 8-15$), with explicit supervision for wrong-branch and outward-too-aggressive cases.

12. Author Contributions and Code Availability

Author contributions. Victor Sebastian Martinez Perez led the project implementation, debugging, experiments, and writing. If additional CS234 or non-CS234 contributors should be credited at co-author level, they should be added here together with a brief description of their role to satisfy the course submission requirement.

Code availability. The code for this project is contained in the project repository used to produce the datasets, checkpoints, and evaluation artifacts described in this report. The final submission should state whether the repository will be public or shared privately with course staff.

13. Suggested Figures

- **Figure 1: System overview.** A diagram showing optimizer-generated data → DT training → autoregressive rollout → projection/fallback/validation → nonlinear solver.
- **Figure 2: Offline loss curves across major runs.** Training and validation curves for the parent improved

run, lateral fine-tune, and failure-conditioned fine-tune, highlighting that offline loss improved even when deployment did not.

- **Figure 3: Baseline vs DT trajectory comparison near first failure.** A no-obstacle and an obstacle example where the DT takes the wrong lateral branch around $k \approx 10$ –15.
- **Figure 4: Acceptance diagnostics by start progress.** Scatter plot of start progress versus fallback count or projection total for the corrected relaxed-gate FT2 run, showing that acceptance clusters in narrow start regions.